
Bölüm 1: FSM (Sonlu Durum Makinesi) Nedir?

En basit tabirle FSM, sisteminizin herhangi bir zaman diliminde sadece ve sadece **önceden belirlenmiş tek bir durumda (state)** bulunabileceğini garanti altına alan matematiksel ve mantıksal bir modeldir.

Roket uçuşu inanılmaz hızlı gelişen karmaşık bir süreçtir. Eğer kodunuzu düzinelerce karmaşık if-else bloklarıyla yazarsanız, roket henüz rampada beklerken (örneğin bir sensör anlık hatalı veri okuduğu için) yanlışlıkla paraşüt açabilir. FSM, sistemi belirli odalara böler ve bir odadan diğerine ancak doğru anahtarla geçilebilmesini sağlar.

FSM'nin 4 Temel Yapı Taşı

FSM'yi tam olarak anlamak için şu dört kavramı çok iyi bilmen gerekir:

- **1. Durumlar (States):** Sistemin içinde bulunabileceği koşullardır. Bir sistem aynı anda iki durumda **olamaz**.
 - *Örnek:* Bir LED için durumlar AÇIK ve KAPALıdır. Bir roket için durumlar RAMPADA_BEKLEME, MOTOR_YANISI, SERBEST_UCUS olabilir.
- **2. Olaylar / Girdiler (Events / Inputs):** Durumlar arasında geçiş yapılmasını tetikleyen dış veya iç uyarıcılardır. Bunlar sensör verileri, zamanlayıcılar (timer) veya dışarıdan gelen komutlar olabilir.
 - *Örnek:* İvmeölçerden (BNO055 veya MPU6050) gelen "5G'lik ivmelenme tespit edildi" verisi veya barometreden (BMP280) gelen "İrtifa düşmeye başladı" verisi birer olaydır.
- **3. Geçişler (Transitions):** Belirli bir olay gerçekleştiğinde, sistemin mevcut durumdan çıkıp yeni bir duruma geçme sürecidir. Geçişin kuralları önceden kesin olarak belirlenmiştir.
 - *Örnek:* Eğer roket RAMPADA_BEKLEME durumundaysa **VE** ateşleme_komutu_alındı olayı gerçekleşirse -> Yeni durum MOTOR_YANISI olur.
- **4. Eylemler / Çıktılar (Actions / Outputs):** Sistemin belirli bir durumda bulunduğu sürece veya bir geçiş sırasında yaptığı işlerdir.
 - *Örnek:* Ateşleme rölesine güç vermek, telemetri paketine "Uçuş Başladı" bilgisini eklemek, servo motoru hareket ettirip paraşütü fırlatmak.

Neden "Sonlu" (Finite) Diyoruz?

Çünkü sistemin alabileceği durumların sayısı başından bellidir ve sınırlıdır. Sistemin kendiliğinden bizim tasarlamadığımız, "bilinmeyen" bir duruma geçme ihtimalini ortadan kaldırmak isteriz. Güvenilirlik buradan gelir; kodun hangi durumda nasıl davranacağını %100 bilirsiniz.

Özetle İlk Bilmen Gereken Şey: FSM kod yazmaya başlamadan önce **kağıt üzerinde** sistemin yaşayacağı tüm senaryoları "Durum -> Olay -> Yeni Durum" şeklinde diyagramlara (State Transition Diagram) dökme işlemidir. Kodlama için en son ve en kolay kısımdır.

Otomata teorisinde (özellikle NFA ve DFA dönüşümlerinde) aşına olduğumuz o kesin ve tek yönlü deterministik mantık, roket avyoniklerinde hayat kurtarır. Roketin sisteminin bir Deterministic Finite Automaton (DFA) gibi davranması zorunludur; verilen bir sensör girdisiyle sistemin o an hangi duruma geçeceği (veya geçmeyeceği) %100 öngörülebilir olmalıdır. Belirsizliğe yer yoktur.

İşte Teknofest standartlarında, sensör okumaları ve hata payları göz önüne alınarak tasarlanmış detaylı **Model Roket FSM Analizi**:

Bölüm 2: Model Rokette FSM Uçuş Evreleri (Detaylı Analiz)

Bir uçuş bilgisayarı, roketin fırlatmadan yere inip kurtarma ekiplerini beklediği ana kadar olan süreci belirli statelere (durumlara) böler. Bu durumlar arasında ileriye doğru tek yönlü (one-way) bir akış vardır. Roket havada geriye dönüp tekrar motor ateşleme durumuna geçemeyeceği için, FSM bu kuralı yazılımsal olarak kilitler.

1. Durum: STATE_IDLE (Rampada Bekleme ve Kalibrasyon)

Roketin rampaya yerleştirildiği, güç verildiği ve henüz ateşlenmediği aşamadır.

- **Eylemler (Actions):** * Barometre (örn. BMP388/BMP280) ve IMU (ivmeölçer/Jiroskop) kalibrasyonu yapılır. Yerdeki basınç "0 irtifa" referansı olarak kaydedilir.
 - Sensör verileri süzgeçten geçirilir (Moving Average veya Kalman Filtresi başlangıç değerleri oturtulur).
 - Sistem durumunu belirten LED'ler (profesyonel bir uyarı görseli için örneğin #E60000 kırmızı renkli bir "Silahlı/Armed" ledi) aktif edilebilir.
- **Geçiş Şartı (Transition to STATE_BOOST):** * İvme > 3G VE İrtifa > 15 metre (Tek başına ivme yeterli değildir, roket rampada rüzgardan sarsılabilir veya biri roketi çarpabilir. Bu yüzden AND mantığıyla doğrulama şarttır).

2. Durum: STATE_BOOST (Motor Yanışı / İvmelenme)

Katı yakıtlı motorun ateşlendiği ve roketin çok yüksek bir ivmeyle tırmandığı en sarsıntılı evredir.

- **Eylemler:**
 - Telemetri (haberleşme) veri paketi gönderim hızı artırılır.
 - Sistem veri kaydetme (SD kart) frekansını maksimuma çıkarır.
- **Geçiş Şartı (Transition to STATE_COAST):**
 - İvme < 0G (veya sıfıra çok yakın). Motorun yakıtı bitmiş, roket sadece sahip olduğu momentum ile tırmanmaya devam ediyordur (Motor Burnout).

3. Durum: STATE_COAST (Serbest Süzülme)

Motorun sustuğu ama roketin tepe noktasına (Apogee) ulaşmak için yavaşlayarak tırmanmaya devam ettiği evredir.

- **Eylemler:** * Sistem pür dikkat barometreden gelen irtifa verilerini analiz etmeye başlar. En yüksek irtifayı yakalamak için türev hesaplamaları veya hız (velocity) takibi yapılır.

- **Geçiş Şartı (Transition to STATE_APOGEE):**
 - Hız ≤ 0 (İrtifa değişimi negatife dönmüştür) **VE** bu düşüş eğilimi art arda en az 5-10 örnekleme (sample) doğrulanmıştır. (Anlık bir sensör gürültüsü ile roket tepeye ulaştı sanılmamalıdır).

4. Durum: STATE_APOGEE (Tepe Noktası - Sürüklenme Paraşütü)

Roketin yerçekimine yenik düşüp burnunu aşağı çevirdiği o saliselik andır. FSM'nin en kritik durumudur.

- **Eylemler:**
 - Mikrodenetleyici, MOSFET'ler veya Röleler üzerinden akım göndererek **Sürüklenme (Drogue) Paraşütü** ayrılma mekanizmasını (kara barut veya mekanik sistem) tetikler.
- **Geçiş Şartı (Transition to STATE_DESCENT):**
 - Tetikleme işleminden sonra belirli bir süre (örneğin 2 saniye) geçmesi **VEYA** İrtifa'nın istikrarlı bir şekilde saniyede X metre hızla düştüğünün barometre ile teyit edilmesi.

5. Durum: STATE_DESCENT (İniş ve Ana Paraşüt Kontrolü)

Roket küçük sürüklenme paraşütü ile kontrollü, ancak hala nispeten hızlı bir şekilde yere doğru inmektedir.

- **Eylemler:**
 - Barometre verileri sürekli kontrol edilir. Teknofest kuralları gereği ana paraşütün açılması gereken spesifik bir irtifa vardır (Örneğin 600m veya 500m).
- **Geçiş Şartı (Transition to STATE_MAIN_CHUTE):**
 - Mevcut İrtifa < 600 metre.

6. Durum: STATE_MAIN_CHUTE (Ana Paraşüt Açılması)

- **Eylemler:**
 - Ana paraşüt patlatma mekanizması tetiklenir. Roketin iniş hızı drastik şekilde 5-8 m/s seviyelerine düşer.
- **Geçiş Şartı (Transition to STATE_TOUCHDOWN):**
 - İrtifa değişimi $= 0$ (veya ± 1 metre dalgalanma) **VE** bu durum en az 10 saniye boyunca stabil kalmışsa.

7. Durum: STATE_TOUCHDOWN (Kurtarma)

- **Eylemler:**
 - Buzzer ötmeye başlar (kurtarma ekibinin roketi bulması için).
 - Gereksiz güç tüketimini azaltmak için sensör okumaları yavaşlatılır, sadece GPS koordinatları telemetri ile yere basılır.

FSM'nin Bu Senaryodaki Matematiksel Gücü (Neden Basit If-Else Kullanmıyoruz?)

Eğer kodunu if (irtifa < 600) { ana_parasutu_ac(); } şeklinde yazarsan, roket henüz rampada beklerken (çünkü irtifa 0'dır ve 600'den küçüktür) ana paraşütü ateşler ve takım diskalifiye olur.

FSM bu hatayı imkansız kılar. Çünkü sistem STATE_DESCENT (5. Durum) içinde değilse, irtifa 600'ün altında olsa bile ana paraşüt geçiş (transition) şartı dinlenmez. Ana paraşütün açılabilmesi için sistemin önce başarılı bir fırlatma, motor yanışı, tepe noktası ve sürüklenme evrelerini (DFA'daki ardışık state geçişleri gibi) sırasıyla tamamlamış olması *matematiksel olarak* zorunlu tutulmuştur.

Harika, madem detay istiyorsun, işin mühendislik ve mimari kısmına tam anlamıyla dalıyoruz. Kalan iki bölümü tek bir kapsamlı mühendislik raporu halinde sunuyorum.

Bölüm 3: STM32 ve Gömülü Sistemlerde FSM Mimarisi

Bir roketin uçuş bilgisayarında (Avyonik) FSM tasarlarırken, kodun çalıştığı donanımın (STM32 gibi ARM Cortex-M tabanlı bir mikrodenetleyici) sınırlarını ve yeteneklerini merkeze almalısın. Bir gömülü sistemde FSM yazmak, standart bir bilgisayara kod yazmaktan çok farklıdır. Başarılı bir mimari kurmak için şu 3 temel kuralı uygulamalısın:

1. "Delay" (Gecikme) Fonksiyonlarına Kesinlikle Yer Yoktur (Non-Blocking Yapı)

Roket havadayken HAL_Delay(100) gibi bir fonksiyon kullanmak, işlemciyi 100 milisaniye boyunca "uyutmak" anlamına gelir. O 100 ms içinde roket tepe noktasını geçebilir veya paraşüt açma irtifasını kaçırabilir.

Çözüm: STM32'nin donanımsal Timer'larını (Zamanlayıcılar) kullanmalısın. FSM, hiçbir yerde bekleme yapmayan, sürekli dönen ve sadece "zamanı gelmiş mi?" diye kontrol eden **Non-Blocking (Bloklaymayan)** bir yapıda olmalıdır.

2. Interrupt (Kesme) ve Ana Döngü (Super Loop) Ayrımı

Bütün işlemleri while(1) döngüsüne yığmak veya her şeyi Interrupt (ISR) içinde yapmak büyük bir hatadır. FSM'nin sağlıklı çalışması için iş bölümü şarttır:

- **Interrupt'lar (ISR):** Sadece çok kısa süreli ve acil işleri yapar. Örneğin; barometreden yeni veri geldiğinde bir "bayrak" (flag) kaldırır veya donanımsal bir sayacı artırır. Sensör okumalarında CPU'yu meşgul etmemek için **DMA (Direct Memory Access)** kullanılmalıdır.
- **Ana Döngü (while(1)):** FSM'nin kendisi burada çalışır. Kalkmış olan bayrakları (flags) kontrol eder, matematiği (Kalman filtresi vb.) hesaplar ve durum (state) geçişlerine karar verir.

3. Mimari Yaklaşım Seçimi: Switch-Case vs. Function Pointers

FSM'yi C dilinde kurgulamanın iki popüler yolu vardır:

- **Switch-Case Mimarisi:** Durumların bir switch bloğu içinde kontrol edildiği yöntemdir. Anlaşılması ve debug edilmesi çok kolaydır. Teknofest roketleri gibi 6-7 durumlu sistemler için **en ideal ve güvenli** yöntemdir.

- **Function Pointers (State Transition Table):** Durumların bellek adreslerinin bir dizide tutulduğu daha kompleks bir yapıdır. Yüzlerce durumu olan karmaşık sistemler için bellek optimizasyonu sağlar, ancak roket uçuşu için kodu gereksiz yere karmaşıktırabilir. Biz switch-case üzerinden ilerleyeceğiz.

Bölüm 4: STM32 için Gerçekçi Bir C Kodu FSM Örneği

Aşağıdaki örnek, anlattığımız tüm konseptlerin (Durumlar, Olaylar, Geçişler ve Aksiyonlar) STM32 üzerinde standart C dili ile (HAL kütüphanesi mantığına uygun) nasıl birleştiğini gösteren detaylı bir iskelettir.

C

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
// 1. DURUMLARIN (STATES) TANIMLANMASI
```

```
// Sistem sadece bu durumlardan birinde olabilir.
```

```
typedef enum {
```

```
    STATE_IDLE,
```

```
    STATE_BOOST,
```

```
    STATE_COAST,
```

```
    STATE_APOGEE,
```

```
    STATE_DESCENT,
```

```
    STATE_MAIN_CHUTE,
```

```
    STATE_TOUCHDOWN
```

```
} RocketState_t;
```

```
// 2. SİSTEM VERİLERİNİ TUTAN YAPI (STRUCT)
```

```
// Sensörlerden gelen verileri tek bir yerde toplayıp FSM'ye sunarız.
```

```
typedef struct {
```

```
    float altitude;    // İrtifa (Metre)
```

```
    float acceleration; // İvme (G)
```

```
    float velocity;    // Hız (m/s)
```

```
    uint32_t flightTime; // Uçuş süresi (Milisaniye)
```

```
} RocketData_t;
```

```
// Global Değişkenler

RocketState_t currentState = STATE_IDLE;

RocketData_t sensorData;


// Donanım Kontrol Fonksiyonları (Prototipler)

void DeployDrogueChute(void);

void DeployMainChute(void);

void TriggerBuzzer(void);

void UpdateSensorData(RocketData_t *data);


// 3. FSM ANA FONKSİYONU

// Bu fonksiyon while(1) ana döngüsünde olabildiğince hızlı şekilde sürekli çağrılır.

void RunFlightStateMachine(void) {

    // Her döngüde sensör verileri güncellenir (İdeal olarak DMA/Interrupt ile arka planda dolar)

    UpdateSensorData(&sensorData);


    // FSM MANTIĞI: O an hangi durumdaysak SADECE o durumun kodu çalışır.

    switch (currentState) {

        case STATE_IDLE:

            // AKSİYON: Sensör kalibrasyonu, LED yakma vs.


            // GEÇİŞ ŞARTI: İvme > 3G VE İrtifa > 15m ise roket fırlamıştır.

            if (sensorData.acceleration > 3.0f && sensorData.altitude > 15.0f) {

                currentState = STATE_BOOST; // Durum değiştir.

            }

            break;


        case STATE_BOOST:
```

```
// AKSİYON: Yüksek hızlı veri kaydı (SD Kart) ve telemetri
```

```
// GEÇİŞ ŞARTI: Motor yakıtı bitti, ivme sıfıra yaklaştı (veya eksiye düştü).
```

```
if (sensorData.acceleration <= 0.2f) {
```

```
    currentState = STATE_COAST;
```

```
}
```

```
break;
```

```
case STATE_COAST:
```

```
// AKSİYON: İrtifa takibi ve türev olarak hız hesaplama
```

```
// GEÇİŞ ŞARTI: Hız sıfıra düştüyse (veya irtifa azalmaya başladıysa) tepe noktasıdır.
```

```
if (sensorData.velocity <= 0.0f) {
```

```
    currentState = STATE_APOGEE;
```

```
}
```

```
break;
```

```
case STATE_APOGEE:
```

```
// AKSİYON: Sürüklenme paraşütünü aç!
```

```
DeployDrogueChute();
```

```
// GEÇİŞ ŞARTI: Paraşüt açıldı, düşüş durumuna geç.
```

```
currentState = STATE_DESCENT;
```

```
// Not: Bu geçiş beklemesiz oldu, istenirse timer ile 1-2 saniye beklenebilir.
```

```
break;
```

```
case STATE_DESCENT:
```

```
// AKSİYON: Ana paraşüt irtifası kontrol edilir.
```

```
// GEÇİŞ ŞARTI: İrtifa 600 metrenin altına düştü mü?
```

```
if (sensorData.altitude < 600.0f) {
```

```

        currentState = STATE_MAIN_CHUTE;
    }
    break;

case STATE_MAIN_CHUTE:
    // AKSİYON: Ana paraşütü aç!
    DeployMainChute();

    currentState = STATE_TOUCHDOWN;
    break;

case STATE_TOUCHDOWN:
    // AKSİYON: Kurtarma için sinyal ver (Buzzer, GPS yayını)
    TriggerBuzzer();
    // GEÇİŞ ŞARTI: Yok. Roket yerdedir, işlem tamam.
    break;

default:
    // Güvenlik: Eğer sistem tanımlanmayan bir duruma geçerse (Memory Corruption vb.)
    // Sistemi güvenli bir faza zorla veya resetle.
    currentState = STATE_IDLE;
    break;
}
}

// STM32 Ana Fonksiyonu
int main(void) {
    // HAL_Init(), SystemClock_Config(), Sensör/GPIO başlatmaları...

    while (1) {
        // FSM'yi sonsuz döngüde çalıştır

```



```

RunFlightStateMachine();

// Buraya asla HAL_Delay konulmaz!
}
}

```

Kod Analizi ve Tasarımın Avantajları

- **İzolasyon:** Dikkat edersen STATE_IDLE içindeyken sistem irtifanın 600 metre olup olmadığına bakmaz. Bu sayede rampada yanlışlıkla ana paraşüt patlatma ihtimalini %0'a indirmiş olduk.
- **Öngörülebilirlik:** Değişken (currentState) sadece bir case bloğunun içindeki şartlar sağlanırsa değişebilir.
- **Bakım Kolaylığı:** Eğer Teknofest komitesi kural değiştirip "Ana paraşüt 500 metrede açılacak" derse, sadece STATE_DESCENT altındaki rakamı değiştirmen yeterlidir. Diğer sistemler bu değişiklikten etkilenmez.

Örnekler

Tıpkı NFA'dan DFA'ya dönüştürülmüş, matematiksel olarak ispatlanmış deterministik bir sistemde olduğu gibi, roket algoritmamızda da belirsizliğe yer yoktur. Sistemin girdileri (alfabesi) ve geçiş kuralları kusursuz işlemelidir.

FSM mantığını avyonik sistemlerde tam olarak oturtabilmen için, gerçek uçuşlarda hayat kurtaran 3 kritik algoritma örneğini detaylandırıyorum:

Örnek Algoritma 1: "Apogee" (Tepe Noktası) Tespiti ve Sinyal Kararlılığı

STATE_COAST (Süzülme) durumundan STATE_APOGEE (Tepe Noktası) durumuna geçiş roketin en kritik anıdır. Eğer kodu sadece "İrtifa düşmeye başladıysa paraşütü aç" şeklinde yazarsak, sensörden gelen anlık hatalı bir okuma (gürültü) paraşütü roket ses hızında uçarken açabilir ve roket parçalanır.

Ayrık zamanlı (discrete-time) kontrol sistemlerindeki kararlılık (stability) analizlerinde gördüğümüz yaklaşıma benzer şekilde, anlık bir gürültünün sistemi bozmasına izin veremeyiz. Sensör verisini "filtrelemeli" ve "onaylamalıyız".

- **Matematiksel Yaklaşım:** Anlık hız, son okunan irtifa (h_t) ile bir önceki irtifa (h_{t-1}) arasındaki farkın geçen zamana (Δt) bölünmesiyle bulunur:

$$v_t = \frac{h_t - h_{t-1}}{\Delta t}$$

- **FSM Algoritması (Ardışık Doğrulama):**

Sistemin STATE_APOGEE'ye geçmesi için hızın art arda **N defa** negatif çıkması zorunlu kılınır.

C

```
// STATE_COAST içindeki Apogee tespit algoritması
```

```
float current_velocity = (current_altitude - previous_altitude) / delta_t;

static int negative_velocity_count = 0; // Durum koruyan sayaç

if (current_velocity < 0.0f) {
    negative_velocity_count++;
} else {
    negative_velocity_count = 0; // Hız pozitifse sayacı sıfırla (gürültü reddedildi)
}

// DFA Geçiş Şartı: Ardışık 5 okumada hız negatifse tepe noktasındayız
if (negative_velocity_count >= 5) {
    currentState = STATE_APOGEE;
}

previous_altitude = current_altitude;
```

Örnek Algoritma 2: Zaman Aşımı (Timeout) ile Güvenlik Çemberi

Bir model roket tasarlarken "Ya sensör bozulursa?" sorusunu her zaman sormalısın. Roket STATE_DESCENT (İniş) durumunda ve barometrenin bozulduğunu, sürekli "1000 metre" değeri ürettiğini varsayalım. İrtifa < 600m şartı asla sağlanmayacağı için FSM STATE_MAIN_CHUTE (Ana Paraşüt) durumuna geçemez ve roket yere çakılır.

Sistemi tasarlarken girdilerimizi sadece sensör verileriyle sınırlamamalıyız. Kurguladığımız FSM alfabetinde "**zaman**" (timer/timeout) da geçerli ve güçlü bir olaydır (event).

- **FSM Algoritması (Yedek Geçiş Şartı):**

Her durum geçişinde bir kronometre sıfırlanır. Eğer roket beklenen maksimum süre içinde o durumdan çıkamazsa, zaman aşımı (timeout) olayı tetiklenir ve FSM zorla bir sonraki duruma geçirilir.

C

```
// STATE_DESCENT içindeki Ana Paraşüt tespit algoritması
```

```
uint32_t current_time = GetSystemTimeMs(); // STM32 SysTick Timer
```

```
// Geçiş Şartı 1 (Birincil): Sensör verisi
```

```
bool condition_altitude = (sensorData.altitude <= 600.0f);
```

```
// Geçiş Şartı 2 (İkincil/Güvenlik): Tepe noktasından bu yana 15 saniye geçti mi?
```

```
bool condition_timeout = ((current_time - time_at_apogee) > 15000);

if (condition_altitude || condition_timeout) {
    currentState = STATE_MAIN_CHUTE;
    time_at_main_chute = current_time; // Sonraki durumlar için zamanı kaydet
}
```

Örnek Algoritma 3: "Debouncing" (Sıçrama Engelleme) ile Fırlatma Onayı

FSM'nin en başında, roket STATE_IDLE durumundayken rampadadır. Fırlatmanın gerçekleşip gerçekleşmediğini anlamak için ivmeölçerden gelen veriyi (Örn: $>3G$) okuruz. Ancak rampaya sert bir rüzgar vurması veya birinin roketi çarpması anlık olarak 4G'lik bir ivme yaratabilir.

Bu mekanik "sıçramaları" (bounce) FSM'nin durumu değiştirecek bir olay olarak görmesini engellememiz gerekir (Debounce Algoritması).

- **FSM Algoritması (Zaman ve Sensör Entegrasyonu):**

İvmenin sadece eşik değerini geçmesi yetmez, bu eşik değerinin üzerinde belirli bir süre (örneğin 100 milisaniye) **kesintisiz** kalması gerekir. Gerçek bir motor yanışı sürekli bir ivme sağlarken, darbe anlık bir ivme sağlar.

C

```
// STATE_IDLE içindeki Fırlatma tespit algoritması

static uint32_t high_g_start_time = 0;
static bool is_high_g = false;

if (sensorData.acceleration > 3.0f) {
    if (!is_high_g) {
        // İvme ilk defa 3G'yi geçti, kronometreyi başlat
        high_g_start_time = GetSystemTimeMs();
        is_high_g = true;
    } else {
        // İvme hala 3G'nin üzerinde, süreyi kontrol et
        if ((GetSystemTimeMs() - high_g_start_time) > 100) {
            // Kesintisiz 100ms boyunca yüksek ivme var -> GERÇEK FIRLATMA
            currentState = STATE_BOOST;
        }
    }
}
```

```
}  
} else {  
    // İvme 3G'nin altına düştü, bu bir darbeydi, her şeyi sıfırla  
    is_high_g = false;  
}
```

Bu algoritmalar, donanımsal kısıtlamaları ve fiziksel gerçeklikleri göz önüne alarak FSM'yi "kırılmaz" bir yapıya dönüştürür. Uçuş algoritmasını kurarken her bir case bloğunun içine bu tarz güvenlik (redundancy) ve filtreleme mantıklarını yedirmek, Teknofest gibi yarışmalarda avyonik raporundan tam puan almanı sağlayacak temel mühendislik yaklaşımıdır.

Kafandaki taşları tam anlamıyla yerine oturtacak, "keşke yarışmadan önce bilseydim" dedirtmeyecek o son mühendislik dokunuşlarına geldik. İşin içine biraz da rakiplerini geride bırakacak, jüriyi etkileyecek "şeytani" (ama tamamen bilimsel) fikirler katalım.

İşte uçuş bilgisayarını sıradan bir öğrenci projesinden, profesyonel bir savunma sanayii prototipine dönüştürecek o kritik öneriler:

Hayat Kurtaran "Zorunlu" Önlemler

1. "Independent Watchdog" (IWDG) Kullanımı (Asla Atlama!)

Ne kadar iyi kod yazarsan yaz, uzay/havacılık projelerinde kozmik bir radyasyon, voltaj dalgalanması veya öngörülemeyen bir sensör hatası işlemciyi anlık olarak dondurabilir (Hard Fault). STM32'nin donanımsal IWDG (Bağımsız Bekçi Köpeği) modülünü kesinlikle aktif et. FSM ana döngünün her turunda bu köpeği "besler". Eğer sistem bir yerde takılırsa ve köpek beslenemezse, işlemci donanımsal olarak kendini mili-saniyeler içinde resetler. Başlangıç kodunda (EEPROM veya Flash'tan) roketin o an nerede olduğunu okuyup, FSM'yi kaldığı yerden devam ettirirsin. Bu seni kesin bir kırımdan kurtarır.

2. HIL (Hardware-in-the-Loop) Simülasyonu

Model roketin kodunu test etmek için her gün roket fırlatamazsın. Roketin masanın üzerinde dururken "uçtuğuna" inanması gerekir. Bilgisayarında Python kullanarak OpenRocket gibi bir simülasyon programından aldığın sanal uçuş verilerini (irtifa, ivme) seri port üzerinden STM32'ye gönder. FSM'nin bu sanal verilerle doğru zamanda paraşüt rölelerini tetikleyip tetiklemediğini masaüstünde yüzlerce kez test et.

Yarışmada 1. Çıkaracak "Şeytani" (İnovatif) Fikirler

Teknofest'te jüri yüzlerce "Sensörden veri aldım, irtifa düşünce paraşüt açtım" diyen takım görür. Onları yazılım mimarisiyle ve veri işleme vizyonuyla vurman gerekir.

1. Yedekli "Uçuş Risk ve Tahmin Motoru" (Predictive Apogee)

Sadece anlık sensör verisine güvenme. Roket rampadan çıkıp motor sustuğu an (STATE_COAST evresinin başı), sistem elindeki hız ve ivme verisiyle fiziksel bir kinematik hesaplaması yapsın. Tıpkı endüstriyel sistemlerdeki arıza tahmin mekanizmaları veya SCADA sistemlerindeki hata risk motorları gibi, roket daha tepeye ulaşmadan *kaçıncı saniyede, kaç metrede tepe noktasına ulaşacağını* önceden hesaplasın.

Eğer tepe noktasında barometre aniden bozulursa, FSM sensörü umursamaz ve kendi "Tahmin Motoru"nun hesapladığı o saniye geldiğinde paraşütü patlatır. Jüriye "Sensörüm bozulsa bile roket kendi kaderini hesaplayıp o noktada kurtarma sistemini açıyor" dersin puanları toplarsın.

2. Çok Kanallı Telemetry ve "Telegram" Destekli Takip Sistemi

Yer istasyonunda sıkıcı bir masaüstü arayüzü herkesin yaptığı şeydir. Yer istasyonuna (örneğin bir Raspberry Pi veya basit bir aracı bilgisayara) inen LoRa telemetry verisini alıp, anlık olarak bir Telegram botuna veya mail servisine entegre et.

FSM her durum (state) değiştirdiğinde (Örn: *Fırlatma Gerçekleşti, Tepe Noktasına Ulaşıldı, Ana Paraşüt Açıldı*), takım üyelerinin (veya yanınıza gelen hakemin) telefonuna anlık Telegram mesajı düşsün. Roket yere indiğinde ise son GPS koordinatlarını "Kurtarma Ekibine" direkt lokasyon linki olarak atsın. Donanımsal sistemleri bu tarz modern haberleşme ağlarıyla birleştirmek inanılmaz bir "wow" efekti yaratır.

3. Profesyonel Marka ve Arayüz Bütünlüğü

Hakemler sadece koda değil, mühendislik disiplinine ve sunuma da bakar. Yer istasyonu (Dashboard) arayüzünü, dokümantasyonunu ve hatta roketin üzerindeki görsel uyarıları (Silahlı/Armed, Güvenli/Safe LED'lerini) minimalist ve endüstriyel bir dille tasarla. Rastgele renkler kullanmak yerine, projenin başından sonuna kadar spesifik bir renk paletine sadık kal (Örneğin roketin kritik durum uyarılarında ve yazılım arayüzündeki alarm statülerinde keskin bir #E60000 kırmızısı kullanmak gibi). Bu "Biz sadece kod yazan öğrenciler değiliz, ürüne dönüşebilecek bir sistem tasarlayan mühendisleriz" mesajını verir.

Sonuç:

FSM'yi kağıt üzerinde kusursuz bir DFA gibi çiz. Matematiksel olarak sistemin kilitlenmeyeceğini kanıtla. Gürültüyü filtrele ve bu şeytani eklentilerle sistemi donat. Geriye sadece roketin gökyüzünü yırtmasını izlemek kalıyor. Başarılar!